



Configurable Web Service

Administration Guide

Document #: D-009082

Last Modified: 8/22/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments	4
1 Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Target Audience	5
2 Introduction to Configurable Web Service.....	5
2.1 Configurable Web Service Overview	5
2.2 Purpose of Configurable Web Service	5
2.3 Process Overview	5
3 Web Service	6
3.1 Creating a New Web Service	6
3.1.1 Creating an Endpoint.....	7
3.1.2 Editing an Endpoint	21
3.1.3 Publishing an Endpoint.....	23
3.2 Versioning a Web Service	24
3.3 Configuring Authorization for a Web Service	25
3.3.1 Creating an API Key	25
3.4 Using a Web Service.....	26
3.4.1 Request URL Format.....	26
3.4.2 Request Authorization	27
3.4.3 Metadata Endpoint	30
3.4.4 Entities	31
3.4.5 Batch Requests	32
3.4.6 Overwriting System Properties.....	38
3.4.7 Uploading Files.....	40

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is important in determining the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation

100 Brickstone Square

Suite 100

Andover, MA 01810

Attention: Aras Technical Documentation

If you want a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support/>

1 Introduction

1.1 Purpose

This document provides detailed information on custom web services and REST APIs for customers implementing applications and integrations with Aras Innovator.

1.2 Scope

This document provides instructions on creating a custom web service and configuring it for use in integrations and custom applications.

1.3 Target Audience

This document is intended for users configuring web services for the Aras Innovator platform. This audience includes Administrators but may also include other users responsible for managing data, such as Process Owners or Business Analysts.

2 Introduction to Configurable Web Service

2.1 Configurable Web Service Overview

The Configurable Web Service (CWS) is a low-code capability for defining and publishing custom REST APIs on the Aras Innovator platform.

2.2 Purpose of Configurable Web Service

The Configurable Web Service enables users to create REST APIs without using SQL, AML, or coding skills. With these REST APIs, users can implement applications and integrations that leverage the data and services of the Aras Innovator platform.

CWS delivers the benefits of the platform's default REST API while providing users with additional control over the data and operations permitted by each custom API.

This capability also includes API Keys for authorizing application requests.

2.3 Process Overview

The process of publishing and using a Configurable Web Service is outlined below:

- Create a new Web Service
- Create an Endpoint for the Web Service
- Configure authorization settings, either API Keys or an OAuth client
- Publish the Web Service
- Send HTTP requests to interact with the REST API produced by the Web Service

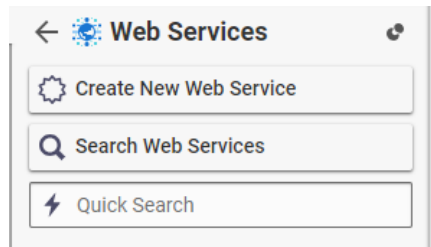
3 Web Service

3.1 Creating a New Web Service

The following steps outline the process of configuring a new Web Service:

1. From the **Table of Contents**, expand **Administrator** and select **External Access**.
2. Expand **External Access** and select **Web Services**.

The **Web Services TOC** appears.



3. Click **Create New Web Service**.

The **Web Service** form appears.

 A screenshot of the 'Web Service 1' configuration form. At the top, there are tabs for 'Web Services' and 'Web Service 1'. Below the tabs are 'Save', 'Done', and 'Delete' buttons. The form is divided into two main sections: 'Web Service' and 'Endpoints'. The 'Web Service' section contains fields for 'Title', 'Endpoint Name' (highlighted in yellow), and 'Description'. The 'Endpoints' section has a sub-tab 'API Keys' and a table with columns 'Active', 'Version', and 'URL'. Above the table are several action buttons: 'Configure Endpoint', 'Create New Version', a search icon, a close icon, a 'Hidden' dropdown, and icons for adding, deleting, and sharing endpoints.

4. Enter the required value in the **Title**, **Endpoint Name**, and **Description** fields.

The screenshot shows the 'Document Share' Web Service configuration. The 'Web Service' tab is active, displaying the following fields:

- Title:** Document Share
- Endpoint Name:** docshare
- Description:** Share Document metadata and related files with applications and integrations

The 'Endpoints' tab is also visible, showing a table with columns for Active, Version, and URL.

Warning The Endpoint Name property is used in the Web Service's URL. This string may include alphanumeric characters and underscores. Do not use spaces or other special characters.

5. Click **Done** to save the changes.
The newly created **Web Service** form appears:

The screenshot shows the 'Document Share' Web Service configuration after saving. The 'Web Service' tab is active, displaying the same fields as before:

- Title:** Document Share
- Endpoint Name:** docshare
- Description:** Share Document metadata and related files with applications and integrations

The 'Endpoints' tab is also visible, showing a table with columns for Active, Version, and URL.

3.1.1 Creating an Endpoint

An Endpoint describes the scope of data, actions, and custom logic allowed for a Web Service. A Web Service must have at least one active Endpoint to “publish” a REST API. A Web Service may also have multiple Endpoints, each with a version number and unique URL. This capability enables users to make significant changes to an API without breaking existing client applications or integrations.

Note: Because a REST API may have multiple active versions, Endpoints do not use the same behavior as other versionable items on the Aras Innovator platform. Refer to section [Versioning a Web Service](#) for more information about creating a new Web Service version.

The following steps outline the process to create a new Endpoint:

1. Open the Web Service.

- Click Edit.

Document Share x

Document Share ☆

Edit

Web Service

Title: Document Share

Endpoint Name: docshare

Description: Document

- Click **Create New Version** in the **Endpoints** section.

Document Share ☆

Save Done Discard

Web Service

Title: Document Share

Endpoint Name: docshare

Description: Share Document metadata and related files with applications and integrations.

Endpoints API Keys

Create New Version

Active	Version	URL ↑

A row will be added to the Endpoints grid.

Endpoints API Keys

Create New Version

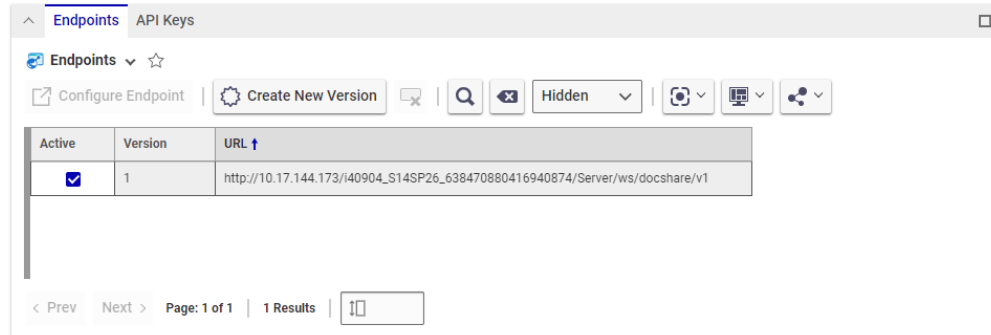
Active	Version	URL ↑
☐		

< Prev Next > Page: 1 of 1 0 Results

4. Click **Save**.

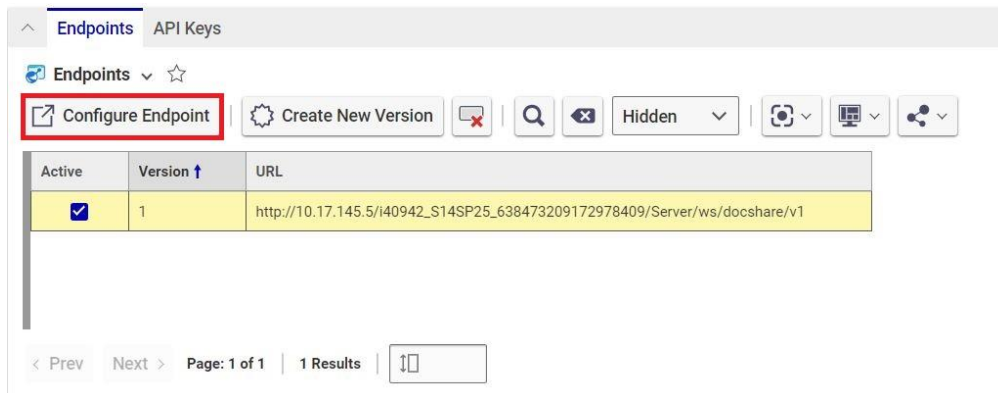
The Version and the endpoint URL will be updated automatically. Each endpoint has a unique URL with the following format:

{Innovator_url}/Server/ws/{endpoint_name}/v{endpoint_version}



5. Select the Endpoint.

6. Click **Configure Endpoint**.

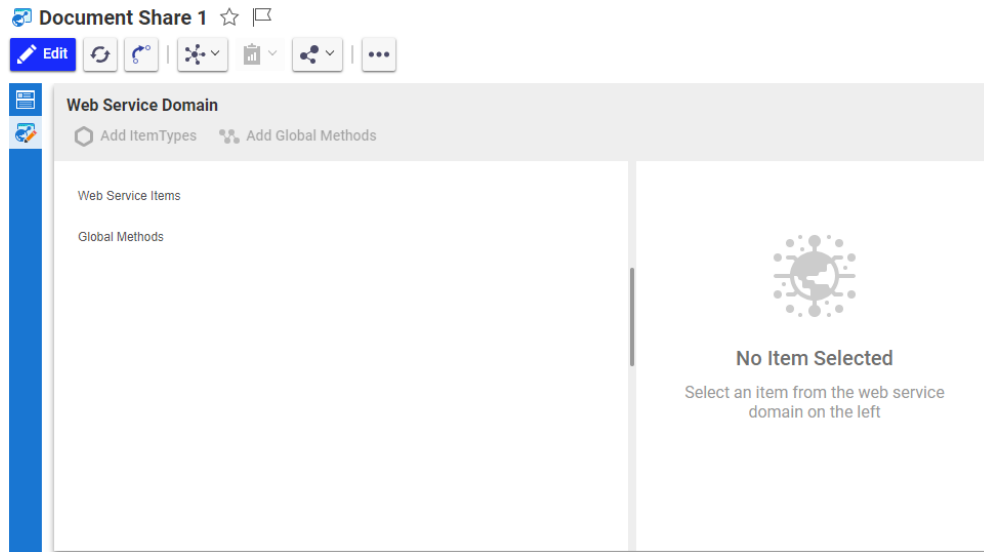


The **Web Service Editor** opens.

The Web Service Editor consists of the following sections:

- The pane on the left lets the user view and modify the Web Service Items (ItemTypes) and Global Methods included in the endpoint's scope.

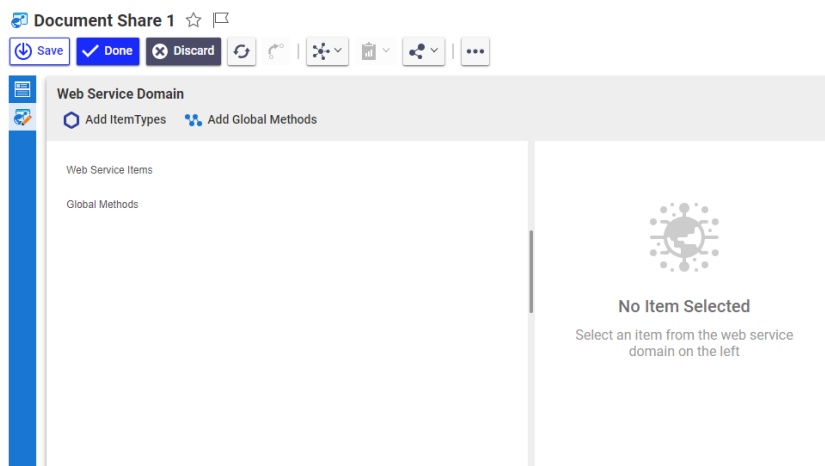
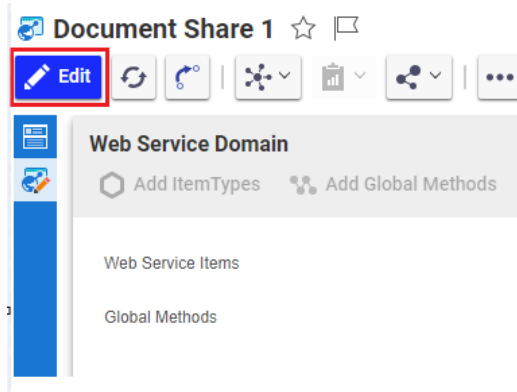
- The right pane enables the user to view and modify settings for any Web Service Item or Global Method selected in the left pane.



3.1.1.1 Adding ItemTypes

The following steps outline the process to add an ItemType to an Endpoint:

1. Click **Edit** in the Web Service Editor.



- Click **Add ItemTypes**.
The **Select Items** dialog box appears.

Select Items

ItemTypes

Simple

	Name ↑	Singular Label	Plural Label	Version...	Depend...	Relatio...	C..	Use Src A...	Descript
▼									

< Prev

Next >

Page: 1

...

25

Download

OK

Cancel

- Click **Search**.
A list of all the ItemTypes displays.

Select Items

ItemTypes

Simple

	Name ↑	Singular Label	Plural Label	Version...	Depend...	Relatio...	C..	Use Src A...	Descript
▼									
	ac_PolicyAccess...	Attribute Query	Attribute Queries	☐	☐	☒	☐	☒	
	ac_PolicyAccessl...	Derived Attribute ...	Derived Attribute ...	☐	☐	☐	☐	☐	
	Access	Access	Permissions	☐	☐	☒	☒	☒	
	Access_ExplicitD...			☐	☐	☒	☐	☒	

< Prev

Next >

Page: 1

...

25

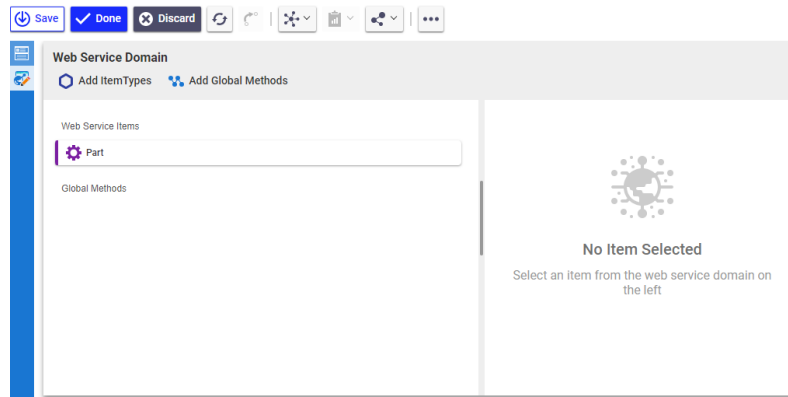
Download

OK

Cancel

4. Select the required ItemType(s) and click **OK**.

The selected ItemType(s) will be added to the Web Service Items list.



Web Service Items may be removed from the Endpoint by selecting the item in the left pane and clicking the Delete button in the header of the right pane. Removing a Web Service Item from an Endpoint does not delete the ItemType from the database.

3.1.1.2 Configuring a Web Service Item

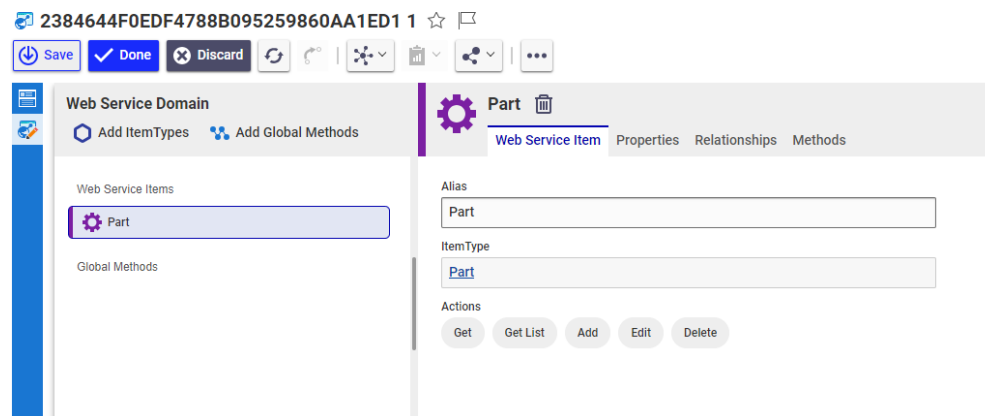
The following steps outline the process to configure a Web Service Item.

1. Click on a Web Service Item in the left pane.

The right pane will display settings for the Web Service Item in four tabs:

- a. **Web Service Item:** Default tab displayed when a Web Service Item is selected
- b. **Properties:** Web Service Item's properties that are available via the Endpoint
- c. **Relationships:** Web Service Item's relationships that are available via the Endpoint
- d. **Methods:** Server Methods that can be run on the Web Service Item via the Endpoint

Web Service Item



1. (Optional) Enter an **Alias**.

This string defines the name of the entity type representing the associated ItemType in the published API. The default value will be the name of the selected ItemType, and all spaces will be replaced with underscores.

Warning Because Alias is used as the entity type in REST API calls, it must be unique and cannot contain spaces or special characters.

- Select the Actions permitted for this Web Service Item.
Get: Request item by id.
- **Get List:** Request item(s) without the id parameter. Requires Get to be selected.
- **Add:** Create a new item.
- **Edit:** Edit an item.
- **Delete:** Delete an item.

Note: Configurable Web Service obeys all access controls configured on the Aras Innovator server. The Action settings for Web Service Items allow admins to *permit or restrict* certain operations for an Endpoint. Enabling an Action on a Web Service Item *does not grant* additional permissions.

Properties

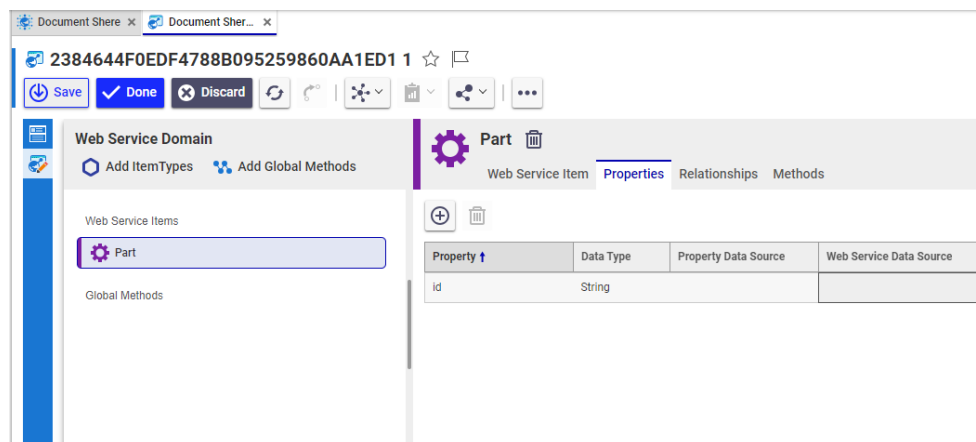
The **Properties** tab displays a table of properties included in the scope of the selected Web Service Item. The column definitions are as follows:

- **Property:** Displays the name of the property
- **Data Type:** Displays the property's data type as defined on the ItemType
- **Property Data Source:** Displays the property's data source as defined on the ItemType
- **Web Service Data Source:** For properties with the "Item" data type, this field identifies a Web Service Item associated with the property's Property Data Source. One will be created automatically if the Endpoint has no corresponding Web Service Item.

Note: The id property is required for all Web Service Items and cannot be removed from the table.

The following steps outline the process to select properties for the Web Service Item:

2. Click the **Properties** tab in the right pane.
3. Click **Add** in the Properties tab.



The Select **Properties** dialog appears.

Select Properties

Name	Label	Data Type	Data Source [...]	Req...	Uni...	Inde...	Fed...	Hid...	Hid...	Key...
classification	Type	String		☐	☐	☐	☐	☐	☐	
config_id		Item	Part	☒	☐	☐	☐	☒	☒	
control_type	Control Type	List		☐	☐	☐	☐	☒	☒	
cost	Cost	Decimal		☐	☐	☐	☐	☐	☒	
cost_basis	Cost Basis	List		☐	☐	☐	☐	☒	☒	
created_by_id		Item	User	☒	☐	☐	☐	☒	☒	
created_on		Date		☒	☐	☐	☐	☒	☒	
css		Text		☐	☐	☐	☐	☒	☒	
current_state		Item	Life Cycle State	☐	☐	☐	☐	☒	☒	

OKCancel

- Select one or more properties.

Select Properties

Name	Label	Data Type	Data Source [...]	Req...	Uni...	Inde...	Fed...	Hid...	Hid...	Key...
classification	Type	String		☐	☐	☐	☐	☐	☐	
config_id		Item	Part	☒	☐	☐	☐	☒	☒	
control_type	Control Type	List		☐	☐	☐	☐	☒	☒	
cost	Cost	Decimal		☐	☐	☐	☐	☐	☒	
cost_basis	Cost Basis	List		☐	☐	☐	☐	☒	☒	
created_by_id		Item	User	☒	☐	☐	☐	☒	☒	
created_on		Date		☒	☐	☐	☐	☒	☒	
css		Text		☐	☐	☐	☐	☒	☒	
current_state		Item	Life Cycle State	☐	☐	☐	☐	☒	☒	

OKCancel

- Click OK.

The newly added properties will be displayed as shown below:

Part 			
Web Service Item			
Properties			
Relationships			
Methods			
			
Property ↑	Data Type	Property Data Source	Web Service Data Source
classification	String		
config_id	String		
control_type	List	Part InvControlType	
cost	Decimal		
created_by_id	Item	User	User
id	String		

Properties may be removed from the Web Service Item by selecting one or more properties and clicking the delete button directly above the table. Removing a property from a Web Service Item does not affect the ItemType definition.

Relationships

The Relationships tab displays the Relationship Types included in the scope of the selected Web Service Item. The column definitions are as follows:

- Relationship Alias:** Name of the RelationshipType
- Related Alias:** Identifies a Web Service Item associated with the relationship's ItemType. One will be created automatically if the Endpoint has no corresponding Web Service Item.

- Click the Relationships tab.
- Click **Add** in the Relationships tab.

Part 	
Web Service Item	
Properties	
Relationships	
Methods	
	
Relationship Alias ↑	Related Alias

The **Add Relationships** dialog appears. It lists the Relationships for the Web Service Item's associated ItemType and the related ItemType.

Add Relationships

×

Relationship	Related
 Part Alternate	 Part
 Part AML	 Manufacturer Part
 Part BOM	 Part
 Part CAD	 CAD
 Part Changes	
 Part Document	 Document
 Part Goal	
 Part MultiLevel BOM	

OK

Cancel

Select one or more Relationships.

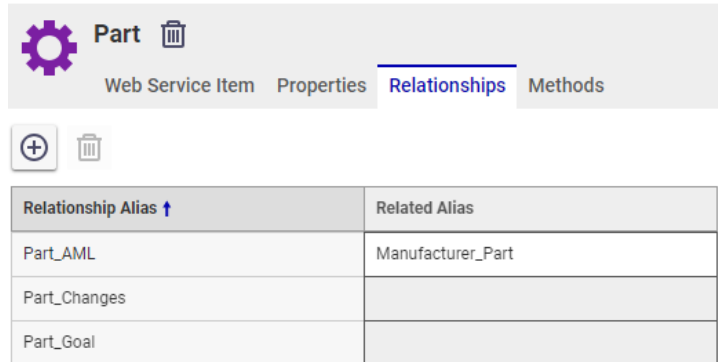
Add Relationships

×

Relationship	Related
 Part Alternate	 Part
 Part AML	 Manufacturer Part
 Part BOM	 Part
 Part CAD	 CAD
 Part Changes	
 Part Document	 Document
 Part Goal	
 Part MultiLevel BOM	

3. Click **OK**.

The newly added relationships will be displayed in the table.



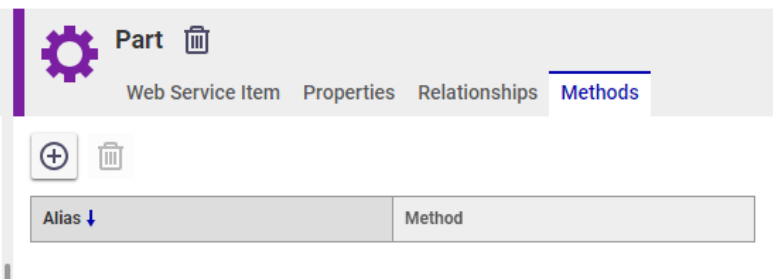
Relationship Alias ↑	Related Alias
Part_AML	Manufacturer_Part
Part_Changes	
Part_Goal	

Relationships may be removed from the Web Service Item by selecting one or more rows and clicking the delete button directly above the table. Removing a relationship from a Web Service Item does not affect the ItemType definition.

Methods

The Methods tab displays the item methods, or “bound methods,” included in the scope of the selected Web Service Item. These should be server Methods explicitly developed for a context item of this ItemType.

Note: All Server Events configured on the ItemType will function as expected for events triggered by REST requests. Those Methods must only be added to the Web Service Item if they should be callable through the Endpoint.



Alias ↓	Method
---------	--------

The column definitions are as follows:

- **Alias:** The name used to execute the Method via the Endpoint
- **Method:** The server Method that will be executed

Warning Because the Method Alias will be used in REST API calls, it must be unique and cannot contain spaces or special characters.

The following steps outline the process of adding Methods:

1. Click the **Methods** tab.
2. Click **Add** in the Methods tab.
The **Select Item** dialog appears.

Click **Run Search**.

Select Items

Methods

Simple

	Name ↑	Method T...	C...	V..	execution_all...	Template [...]	Comments
▼		▼					

A list of items will be displayed.

3. Select one or more items.

Select Items

Methods

Simple

	Name ↑	Method T...	C...	V..	execution_all...	Template [...]	Comments
▼	Run	CSharp ▼					
	RunChangeReport	CSharp	☐		Administrators		

4. Click OK.

The newly added records will be displayed.

Part

Web Service Item Properties Relationships **Methods**

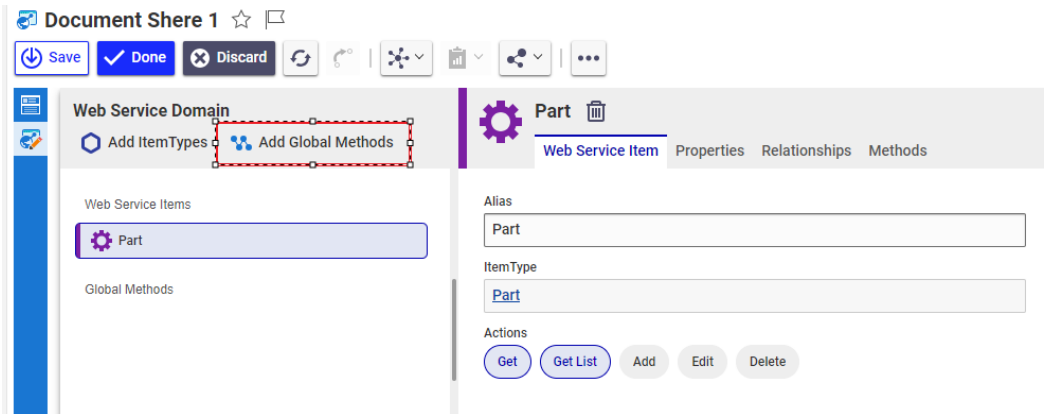
Alias	Method
RunChangeReport	RunChangeReport
bomrollup	bomrollup

Methods may be removed from the Web Service Item by selecting one or more rows and clicking the delete button directly above the table. Removing a Method from a Web Service Item does not delete the Method from the database or affect the ItemType definition.

3.1.1.3 Adding Global Methods

Global Methods, or “unbound methods”, are server Methods that can be executed independently of a specific context item type.





The column definitions are as follows:

- **Alias:** The name used to execute the Method via the Endpoint
- **Method:** The server Method that will be executed

Warning Because the Method Alias will be used in REST API calls, it must be unique and cannot contain spaces or special characters.

The following steps outline the process to add Global Methods:

1. Click **Add Global Methods** directly above the left pane.

The **Select Item** dialog appears.

Click **Run Search**.

Select Items

Methods

Search

Simple

	Name	Method T...	C...	V...	execution_all...	Template [...]	Comments

< Prev

Next >

Page: 1

25

Download

OK

Cancel

2. Click Search.

A list of all the corresponding Items will be displayed.

Select Items

Methods

Q

X

Simple

	Name ↑	Method T...	C...	V..	execution_all...	Template [...]	Comments
▼		▼					
	ac_BeforeChangeItemAttri...	Internal	☐	1	Administrators		
	ac_ClearTargetPropOnEdit...	JavaScript	☐	1	Administrators		
	ac_OnBeforeChangeAttrD...	Internal	☐	1	Administrators		
	ac_OnBeforeDeleteAttrDe...	CSharp	☐	1	Administrators		
	ac_PreventChangeApplied...	JavaScript	☐	1	Administrators		

< Prev

Next >

Page: 1

...

25

Download

OK

Cancel

3. Select one or more server Methods.

Select Items

Methods

Q

X

Simple

	Name ↑	Method T...	C...	V..	execution_all...	Template [...]	Comments
▼		▼					
	ac_BeforeChangeItemAttri...	Internal	☐	1	Administrators		
	ac_ClearTargetPropOnEdit...	JavaScript	☐	1	Administrators		
	ac_OnBeforeChangeAttrD...	Internal	☐	1	Administrators		
	ac_OnBeforeDeleteAttrDe...	CSharp	☐	1	Administrators		
	ac_PreventChangeApplied...	JavaScript	☐	1	Administrators		

< Prev

Next >

Page: 1

...

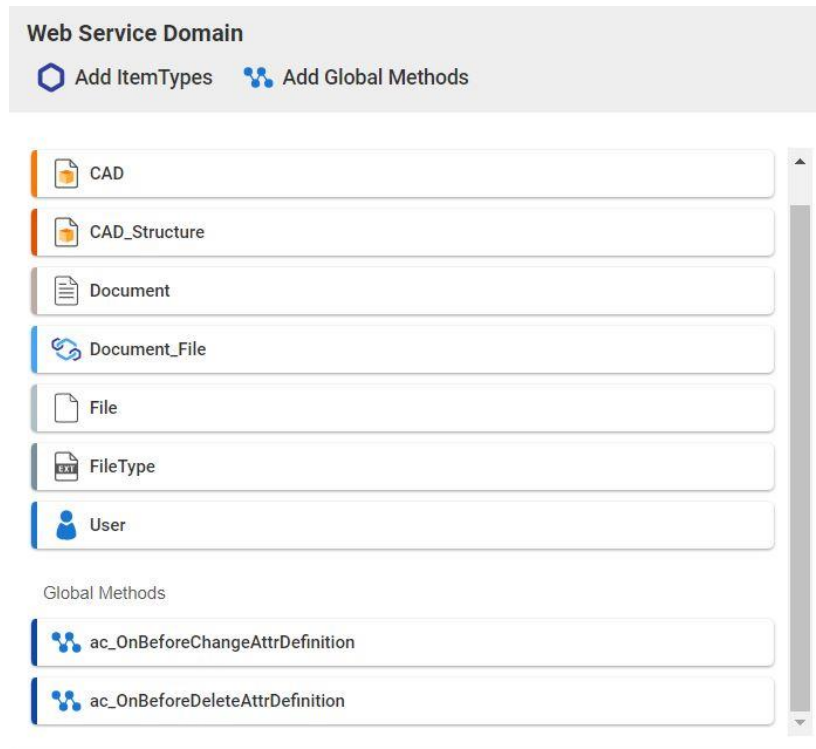
25

Download

OK

Cancel

4. Click **OK**.
The newly added method(s) are in the left pane.



5. Click a **Global Method** in the left pane.

The right pane will display the Global Method's settings in a single tab.

- **Alias:** The name used to execute the Method via the Endpoint
- **Method:** The server Method that will be executed
- **Parameters:** This table enables admins to define named parameters that may be passed in the body of a request for use in the server Method code

Warning Because the Method Alias will be used in REST API calls, it must be unique and cannot contain spaces or special characters.

Global Methods may be removed from the Endpoint by selecting the Method in the left pane and clicking the Delete button in the header of the right pane. Removing a Global Method from an Endpoint does not delete the Method from the database.

6. Click **Done** to save the Endpoint.

3.1.2 Editing an Endpoint

The following steps outline the process to create a new Endpoint:

1. Open the Web Service.
2. Click **Edit**.

Document Share x

Document Share ☆

Edit

Web Service

Title

Document Share

Endpoint Name

docshare

Description

Document

3. Select the Endpoint by clicking on it.
4. Click Configure **Endpoint**.

Endpoints API Keys

Endpoints ☆

Configure Endpoint

Create New Version

Hidden

Active	Version ↑	URL
☒	1	http://10.17.145.5/i40942_S14SP25_638473209172978409/Server/ws/docshare/v1

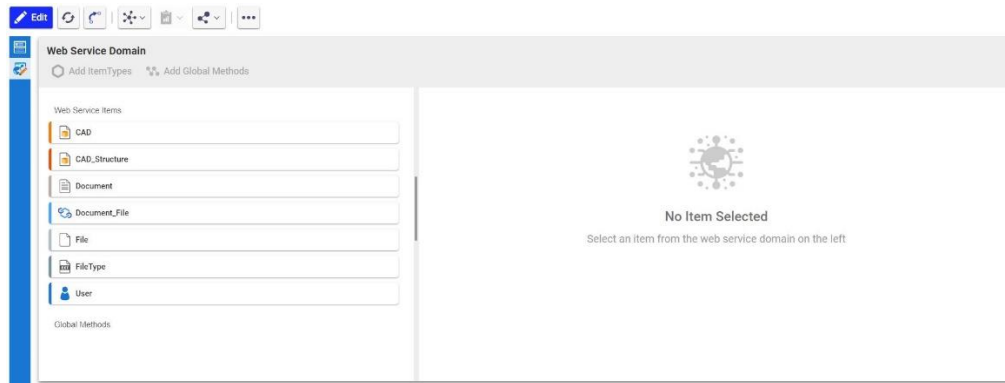
< Prev

Next >

Page: 1 of 1

1 Results

The Endpoint opens in the **Web Service Editor**.

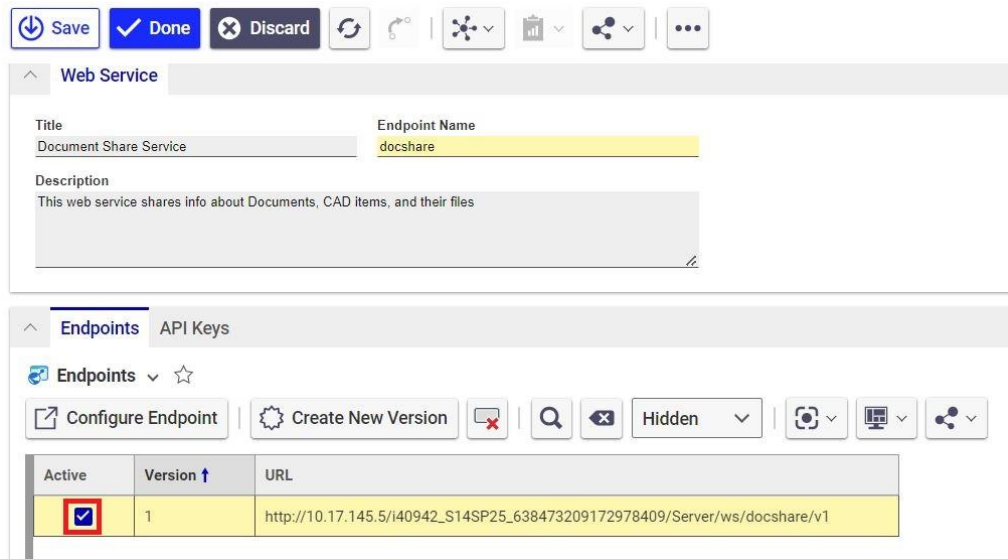


5. Click **Edit** in the Web Service Editor.
6. Edit the **Endpoint** as needed, following the appropriate steps from Sections 3.1.1.1 “Adding ItemTypes” and 3.1.1.2 “Configuring a Web Service Item”.
7. Click **Done** in the Web Service Editor when all edits are complete.

3.1.3 Publishing an Endpoint

Creating an Endpoint does not automatically make the REST API accessible. It must be published or made “active” before it can respond to requests. The following steps outline the process for publishing an Endpoint.

1. Open the Web Service.
2. Click **Edit**.
3. In the Endpoints grid, enable the Active checkbox for the Endpoint to be published.



4. Click **Done** in the Web Service toolbar.

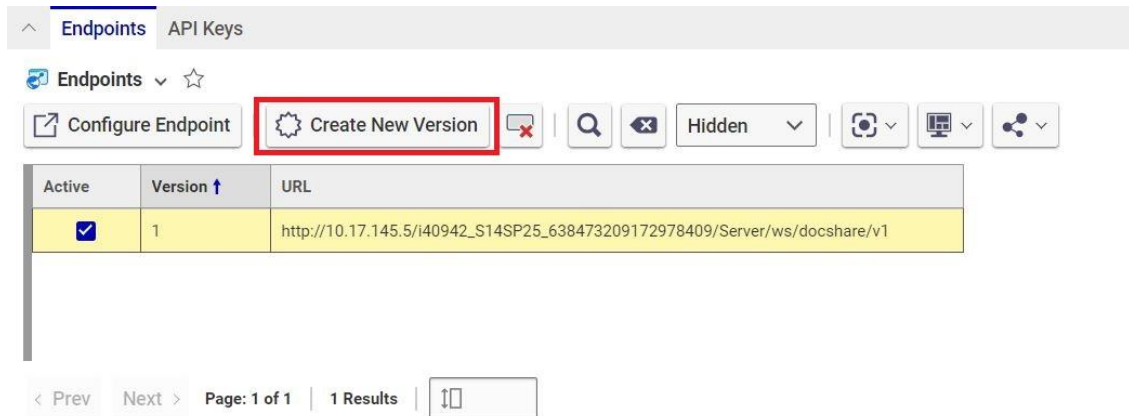
The Endpoint is now published and can respond to HTTP requests. The Endpoint can be unpublished again by disabling the Active checkbox.

Note: The Active property can also be managed on the Endpoint form.

3.2 Versioning a Web Service

A Web Service may have multiple Endpoints, each describing a version of the Web Service. This enables the user to make significant changes to an API without breaking existing client applications or integrations. The following steps outline the process of versioning a Web Service with a new Endpoint.

1. Open the Web Service.
2. Click **Edit**.
3. In the Endpoints grid, select the Endpoint that will be the basis of the new version.
4. Click the **Create New Version** button in the grid toolbar.

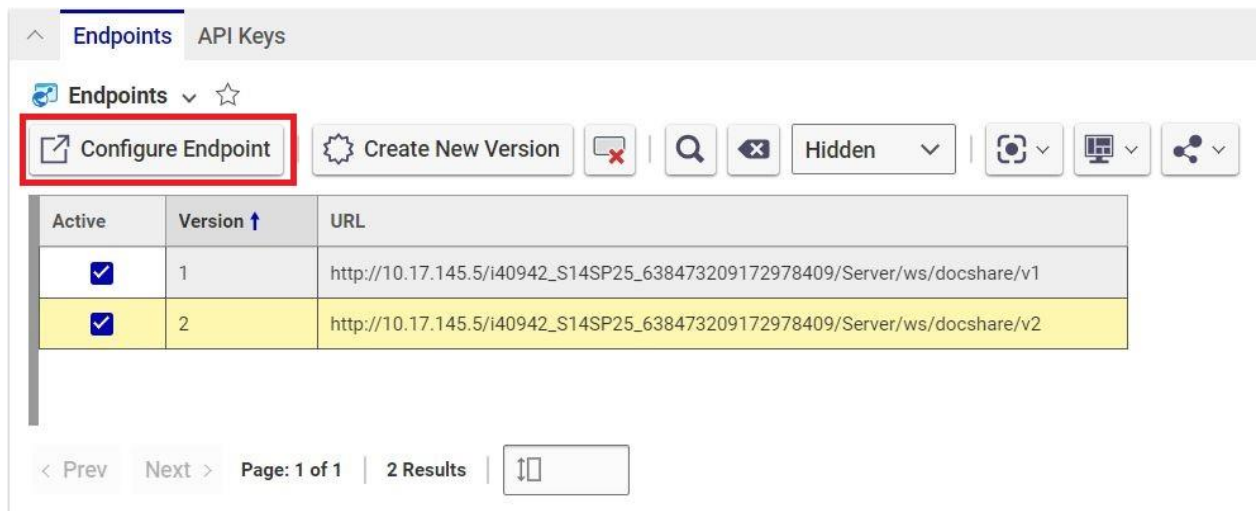


The screenshot shows the 'Endpoints' tab in a web service configuration interface. The toolbar includes buttons for 'Configure Endpoint', 'Create New Version' (highlighted with a red box), and several utility icons. Below the toolbar is a table with one endpoint.

Active	Version ↑	URL
☒	1	http://10.17.145.5/i40942_S14SP25_638473209172978409/Server/ws/docshare/v1

At the bottom, there is a pagination bar showing 'Page: 1 of 1' and '1 Results'.

5. Click **Save** in the Web Service toolbar.
6. In the Endpoints grid, select the Endpoint with the new version.
7. Click **Configure Endpoint** in the grid toolbar.



The screenshot shows the 'Endpoints' tab after creating a new version. The toolbar now includes a 'Configure Endpoint' button (highlighted with a red box) and the 'Create New Version' button. The table now contains two endpoints.

Active	Version ↑	URL
☒	1	http://10.17.145.5/i40942_S14SP25_638473209172978409/Server/ws/docshare/v1
☒	2	http://10.17.145.5/i40942_S14SP25_638473209172978409/Server/ws/docshare/v2

The pagination bar at the bottom shows 'Page: 1 of 1' and '2 Results'.

8. Edit the Endpoint as needed. Then Save the Web Service and publish the newest Endpoint version.

Note: A Web Service may have more than one published or “active” Endpoint, each with a unique url.

3.3 Configuring Authorization for a Web Service

3.3.1 Creating an API Key

An API Key enables the Aras Innovator server to authorize requests and execute the request as the User associated with the key. API Keys are most commonly assigned to a service User that is used for server-to-server requests or other scenarios where an end user cannot be prompted for credentials.

Note: Applications and integrations that execute requests as a specific end user should use the OAuth credential flow instead of API Keys. This is the same authorization approach supported by the platform's default REST API.

API Keys contain the following properties:

- **Name:** Name of the API Key
- **User:** The User that will be used for all requests made with this key. If a user is not entered, a system user will be automatically generated.
- **Description:** Description of the API Key
- **Created By:** The User who created the API key
- **Created On:** When the API key was created
- **Scope:** Optional. Selecting “Override System Properties” will enable requests using this API Key to overwrite the data in system properties.

Warning Only enable the “Override System Properties” scope when a use case requires an application or integration to overwrite the system property values in the Aras Innovator database. This feature may replace critical metadata such as created_by_id, modified_by_id, created_on, modified_on, state, etc. See the section “Overwriting System Properties” below for more detail on this capability.

Warning After generating the API Key, copy and store it securely, as it will not be visible again.

Note: System Users generated for API Keys are not automatically added to any group identities or access control configurations. Be sure to check that generated system users have permission to perform the operations described in the Endpoint.

The following steps outline creating an API Key for a Web Service.

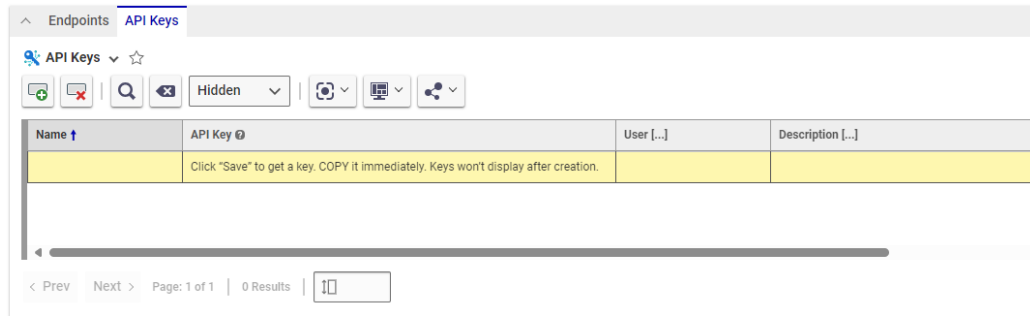
1. Open the Web Service.
2. Click **Edit**.

Click the **API Keys** tab.



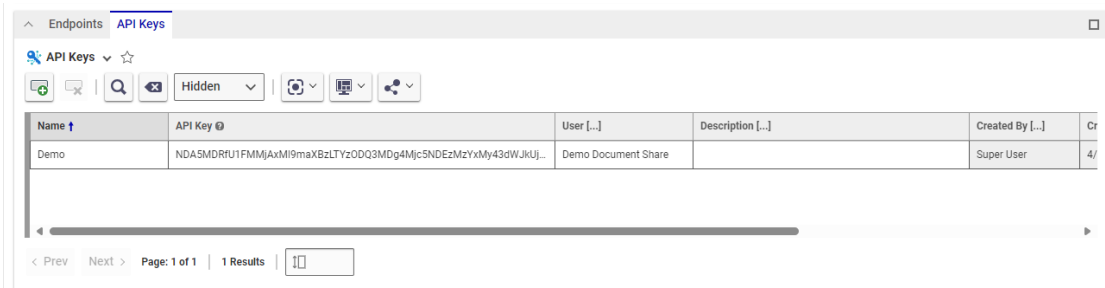
3. Click the **New API Key** button.

A new record will be added to the table, as shown below:



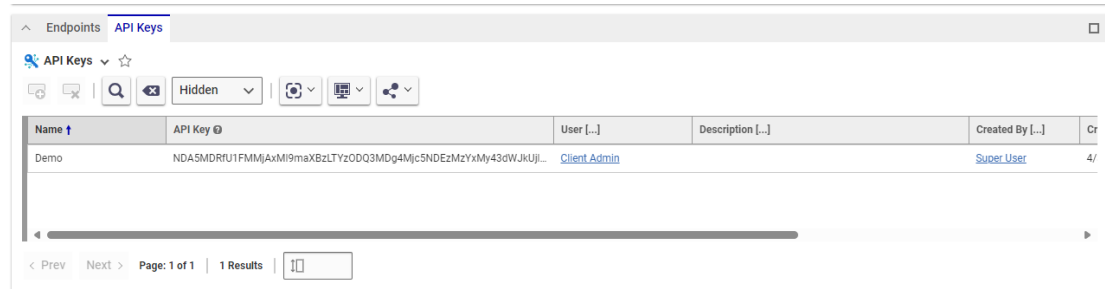
Name ↑	API Key @	User [...]	Description [...]
	Click "Save" to get a key. COPY it immediately. Keys won't display after creation.		

4. Enter **Name**.
5. (Optional) Enter the **User** that will be used for all requests made with this API Key.
If a User is not selected, one will be generated. If using a generated user, be sure to grant that user permission for the requests that will be made with the API Key.
6. Click **Save**.
Saving the record will display the API key. Save the key, as it will not be visible after creation.



Name ↑	API Key @	User [...]	Description [...]	Created By [...]	Cr
Demo	NDA5MDRfU1FMMjAxMj9maXBzLTZzODQ3MDg4Mjc5NDEzMzYxMy43dWJkUj...		Demo Document Share	Super User	4/

7. Click **Done**.
The newly added **API key** will be displayed.



Name ↑	API Key @	User [...]	Description [...]	Created By [...]	Cr
Demo	NDA5MDRfU1FMMjAxMj9maXBzLTZzODQ3MDg4Mjc5NDEzMzYxMy43dWJkUj...	Client Admin		Super User	4/

3.4 Using a Web Service

The Configurable Web Service produces REST APIs that follow the same OData protocol used by the platform's default REST API. As a result, sending requests and receiving responses from a CWS Endpoint is very similar to the default REST API.

This section will outline steps that are unique to using a CWS Endpoint and provide references to the RESTful API documentation for steps shared with the default API.

3.4.1 Request URL Format

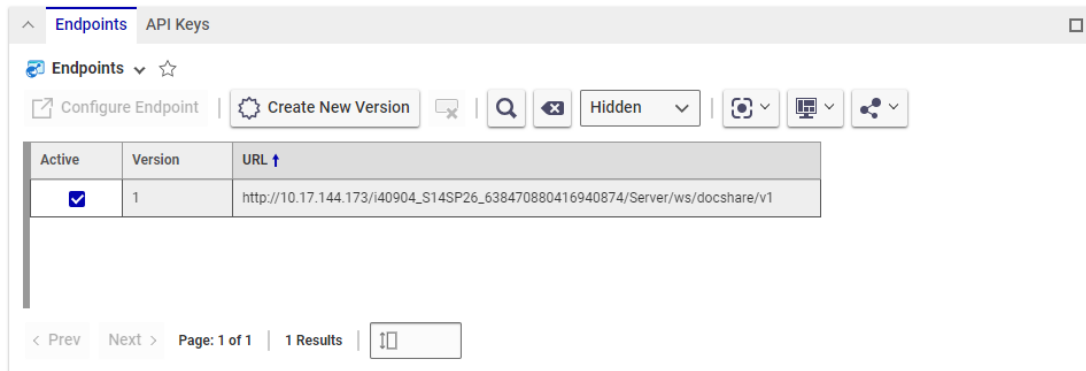
Each Aras Innovator instance has one endpoint for the default REST API. This default endpoint URL uses the following format:

```
{Innovator_url}/Server/odata/{...}
```

However, CWS can produce multiple REST APIs – each with a unique endpoint URL in the following format:

```
{Innovator_url}/Server/ws/{endpoint_name}/v{endpoint_version}/{...}
```

To confirm the Endpoint URL(s) for a REST API published via CWS, open the Web Service item and check the Endpoints tab.



3.4.2 Request Authorization

All requests sent to the Aras Innovator server must be properly authorized. For the platform's default REST API, that means an application must request an access token from the OAuth server and include it in the headers of each HTTP request. CWS supports both OAuth access tokens and API Keys.

The following sections outline how to use each authorization approach in a request to a CWS Endpoint.

3.4.2.1 OAuth Token

The process for using an OAuth access token with a CWS Endpoint is the same as the process for the default REST API.

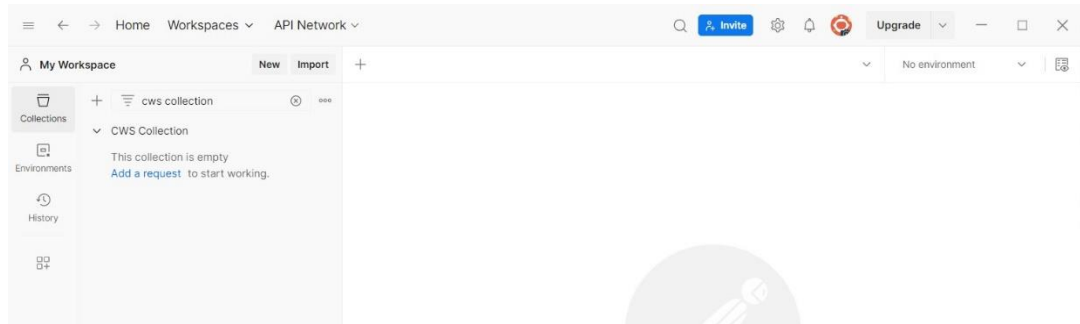
Note: Refer to section **Using OAuth 2.0 Tokens from the Authentication Server** in the Aras Innovator REST API documentation for step-by-step instructions.

3.4.2.2 API Key

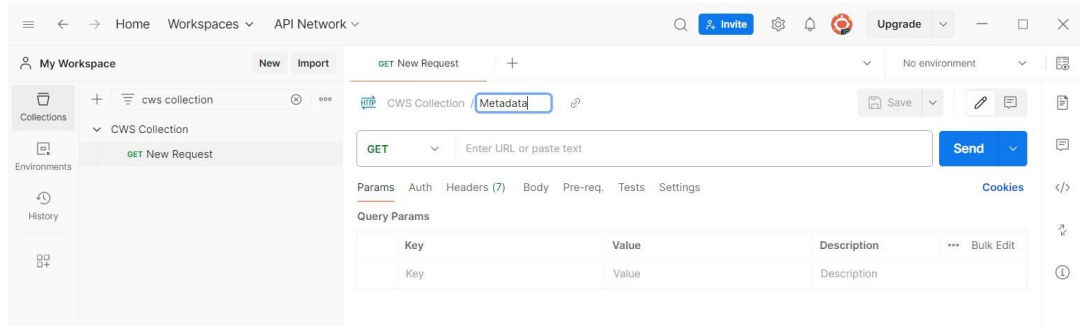
The following steps outline the process of using an API Key to authorize a request to a CWS Endpoint.

Note: This example uses the Postman HTTP client on the desktop. However, the general concept of passing the API Key in the request's Authorization header applies to any HTTP request.

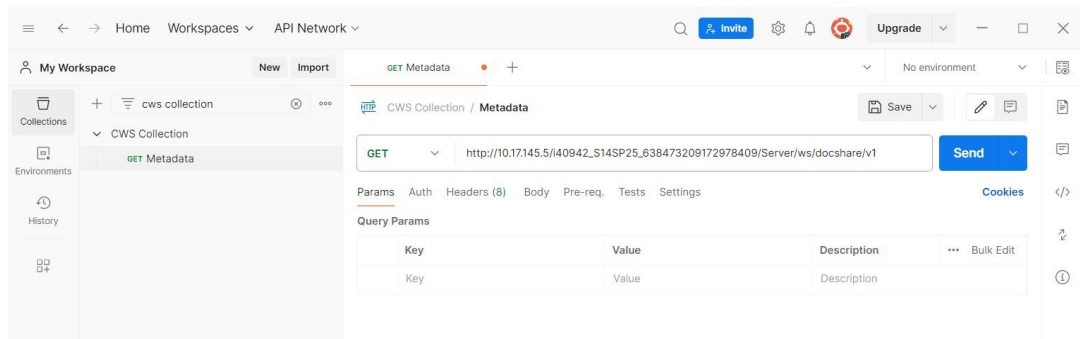
1. Download the Postman application on the local machine.
2. Under **My Workspace**, select **Collections** and click **Create New Collection**.
3. Provide a **name** for the new Workspace. For example, "CWS Collection".



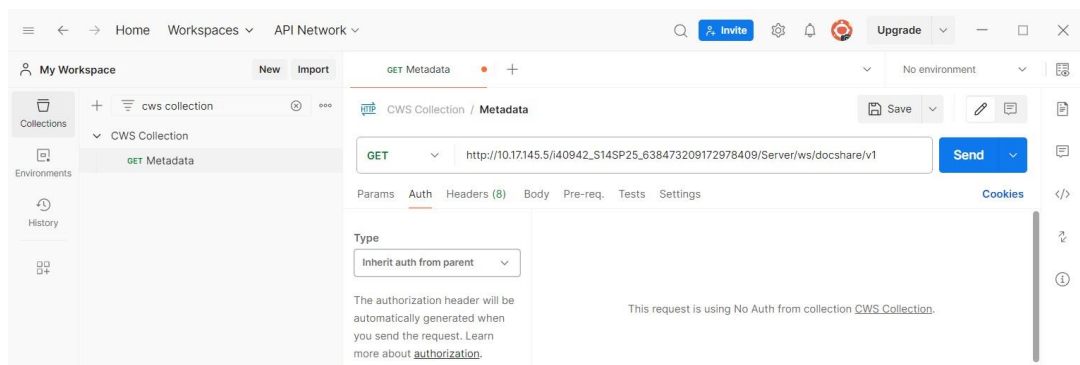
4. Click **Add a request** under the new Collection.



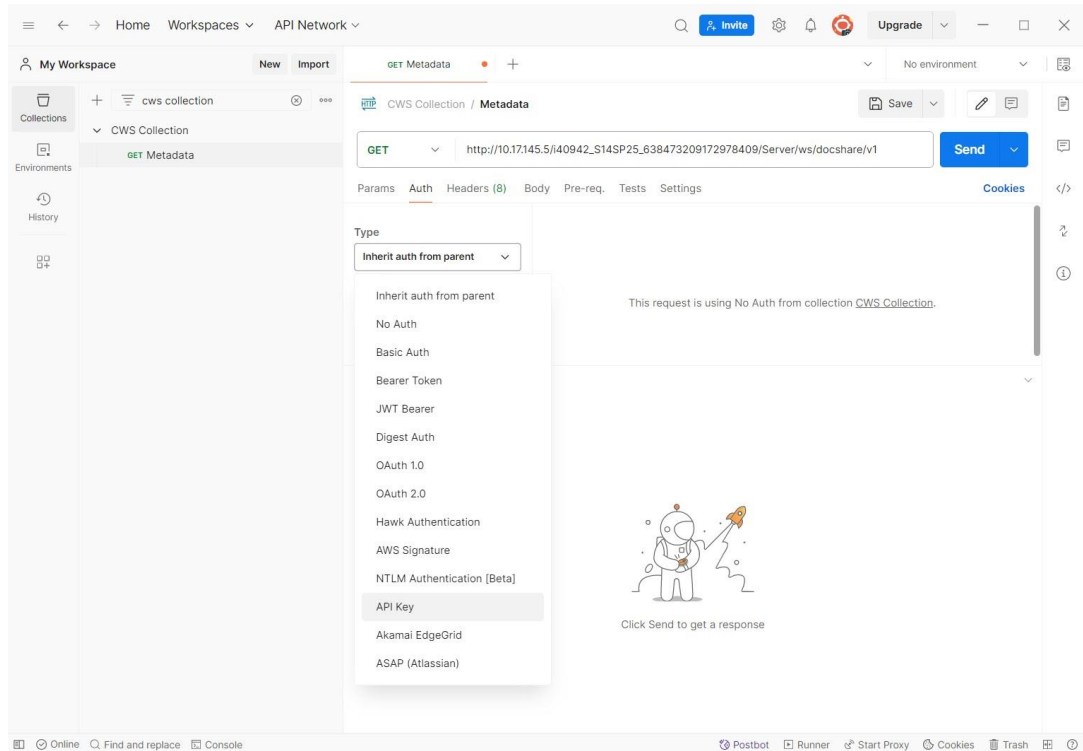
5. Provide a **name** for the new Request. For example, “Metadata”.
6. In the address bar, enter the CWS **Endpoint URL**.



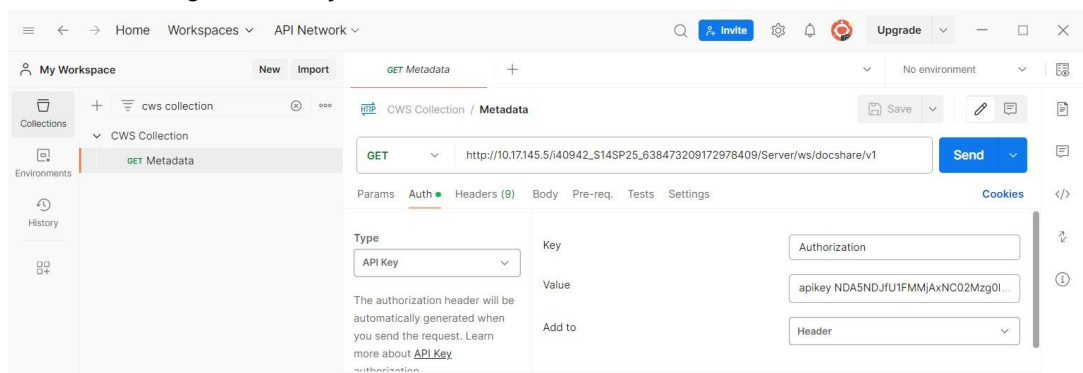
7. Click the **Auth** tab.



8. Select **API Key** from the Type dropdown.



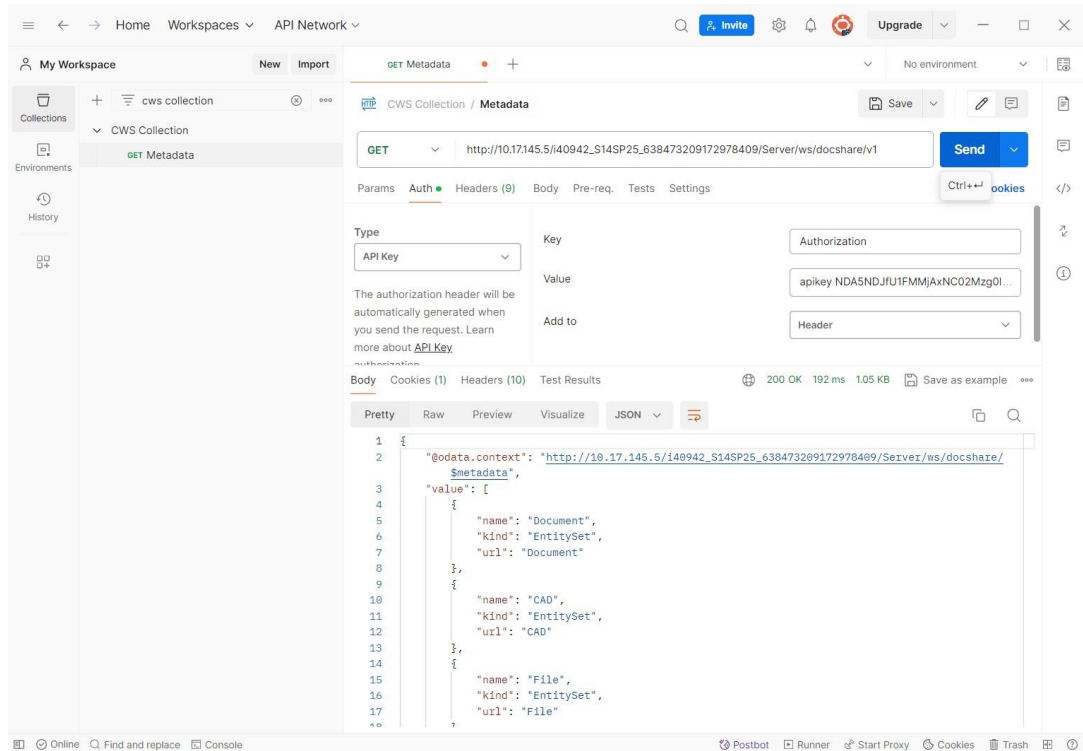
9. In the Value field, enter “**apikey**” followed by an API Key generated with the steps in section Creating an API Key.



Note: It's important to include a space between the “apikey” prefix and the key's value.

10. Click **Send**.

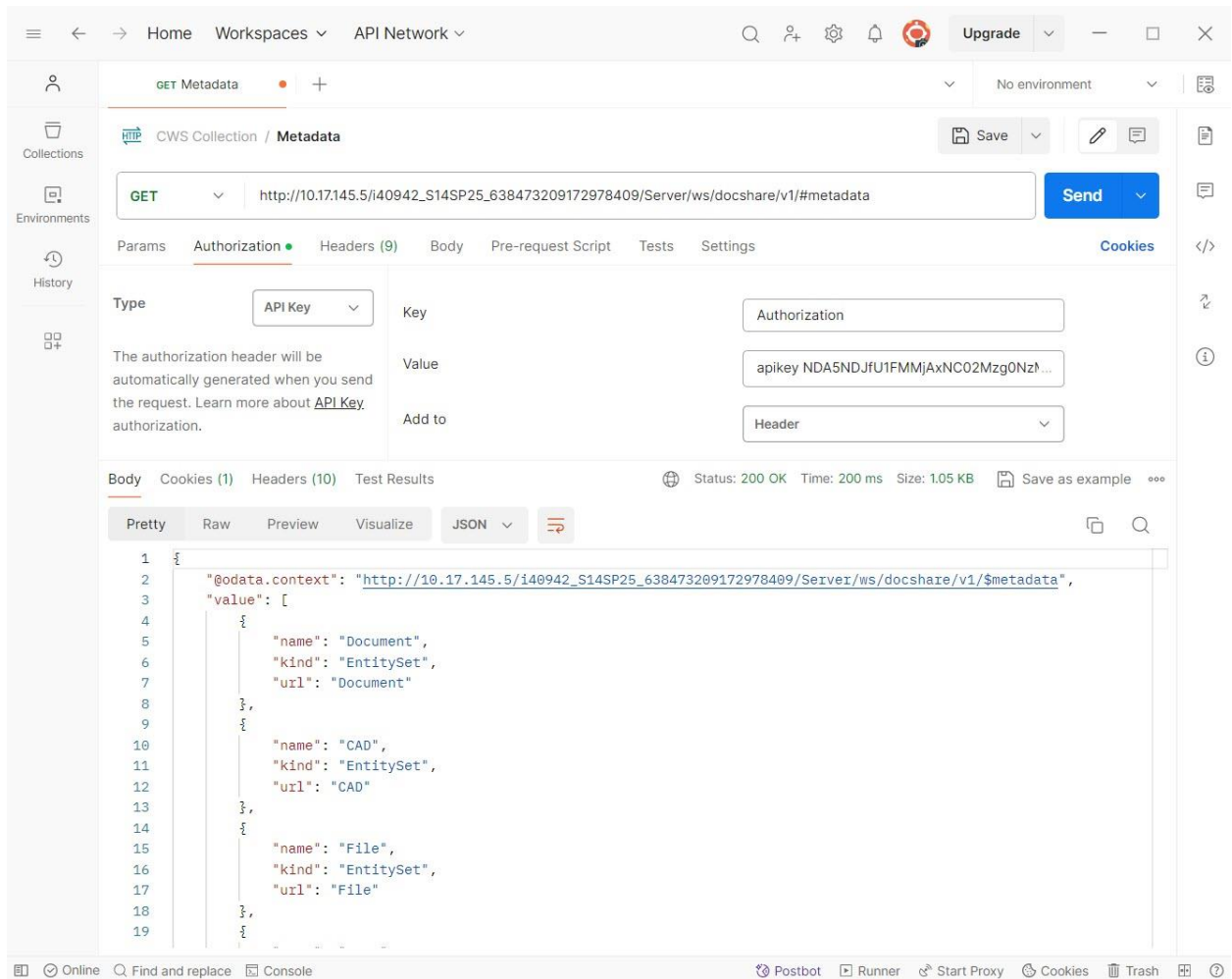
The request should return a 200 OK response and the response body will appear in the bottom pane of the Postman application.



Note: The Authorization header is the default location for passing an API key in a CWS request. However, the server will check other non-reserved headers for a valid API key with the prefix "apikey_" if the Authorization header of a request is not populated.

3.4.3 Metadata Endpoint

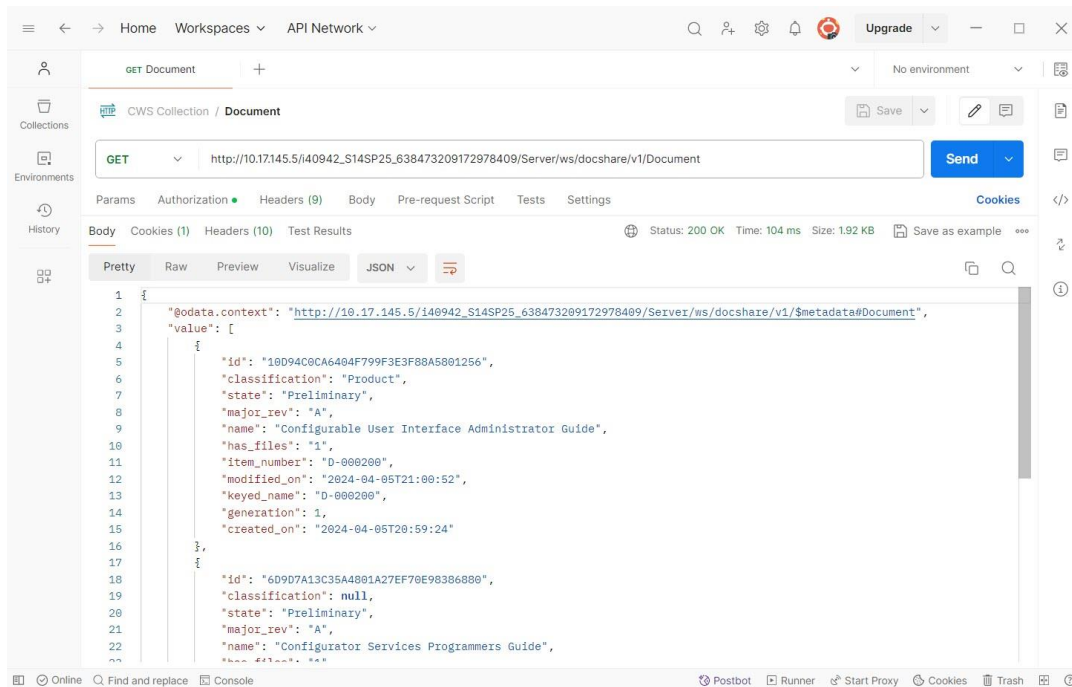
REST APIs published with CWS have a special metadata endpoint that describes the API's schema in an Odata-compliant format. This schema can be accessed with the Endpoint url as shown in the previous section. It can also be accessed by appending #metadata to the Endpoint url.



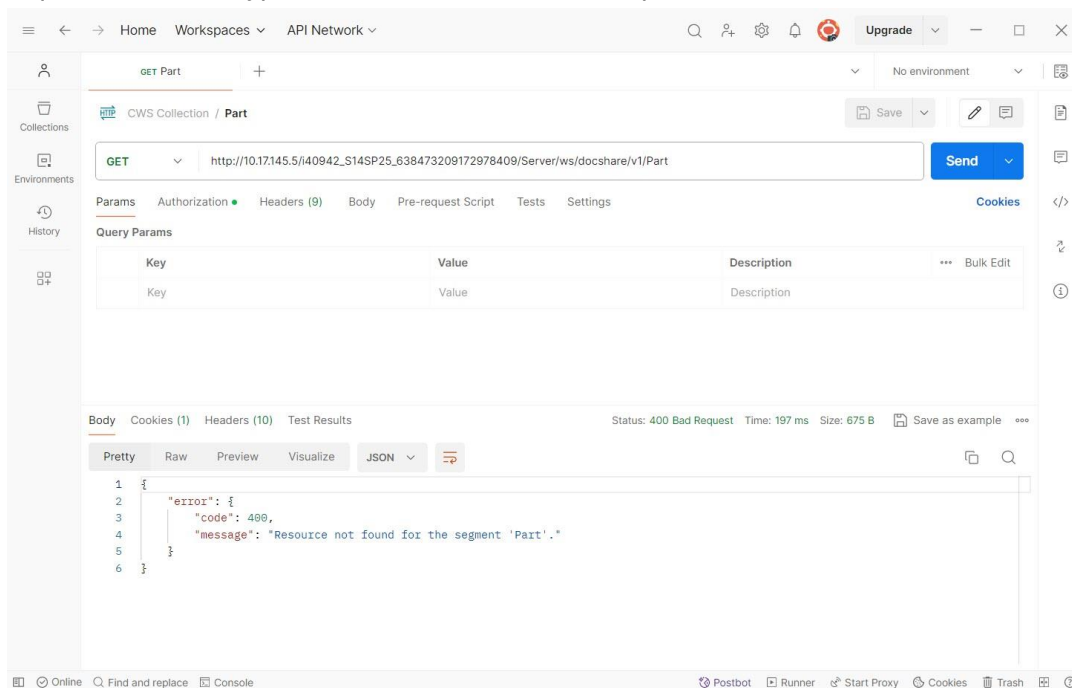
3.4.4 Entities

The scope of the platform's default REST API includes all ItemTypes and server Methods in the Aras Innovator database. This contrasts with the scope of CWS-published APIs, which include only the data and logic described in the Endpoint configuration.

For example, CWS Endpoint that permits “get all” for Documents will respond with a 200 OK status and Document data in the response body.



However, the same CWS Endpoint will respond with a 400 Bad Request status if it receives a request for an ItemType that is not included in the scope.



3.4.5 Batch Requests

In OData, batching enables developers to bundle multiple requests into a single call to the `$batch` endpoint (see [OData documentation](#)). A batch request is represented as a Multipart MIME message, a standard format allowing the representation of multiple parts, each of which may have a different content type, within a single request.

Batch requests are submitted as a single **HTTP POST** request to the \$batch endpoint of a Web Service. Any requests nested in a batch are executed sequentially and synchronously.

3.4.5.1 Batch Request Headers

This section describes the headers for an HTTP POST request to the \$batch endpoint.

- **Content-Type:** Required. This header must have the *multipart/mixed* value and a boundary specification as defined in the Batch Request Body section (see an example below).

Key	Value	Description
☒ DATABASE	{{DATABASE}}	
☒ AUTHUSER	{{AUTHUSER}}	
☒ AUTHPASSWORD	{{AUTHPASSWORD}}	
☒ Content-Type	multipart/mixed; boundary=batch_abcdef8-fc75-4b56-6c71-56071383e77b	

- **OData-Version:** Optional. If using this header, set the value to 4.0.
- **Prefer:** Optional. By default, the CWS engine stops processing a batch request when any of the nested requests return an error. To continue processing the batch in case of an error in a nested request, include the Prefer header with the value *odata.continue-on-error*.

3.4.5.2 Batch Request Body

The body of a batch request comprises a series of individual requests and [Change Sets](#), each represented as a distinct MIME part (i.e. separated by the boundary defined in the *Content-Type* header).

An individual request in the context of a batch request can be

- [Data request](#);
- [Data Modification](#) request;
- [Action invocation](#) request.

A MIME part representing an individual request must include a *Content-Type* header with the value *application/http* and should contain a *Content-Transfer-Encoding* header with the value *binary*.

CWS supports three Request-URI formats of HTTP requests serialized within MIME part bodies:

- Absolute URI with schema, host, port, and absolute resource path
 - The absolute URI of each request in a batch must be the same as the URI of the source batch request.
- Absolute resource path and separate Host header
- Resource path relative to the batch request URI

The following sample batch request demonstrates each of these formats.

```

POST http://{{HOST}}/{{alias}}/Server/ws/CWS_OdataSet/v1/$batch

Params Authorization Headers (15) Body Pre-request Script Tests Settings
none form-data x-www-form-urlencoded raw binary GraphQL Text

1 --batch_abcdeff8-fc75-4b56-8c71-56071383e77b first boundary
2 Content-Type: application/http
3
4 GET http://{{HOST}}/{{alias}}/Server/ws/EndpointName/v1/Part HTTP/1.1 absolute URI with full path
5
6
7 --batch_abcdeff8-fc75-4b56-8c71-56071383e77b
8 Content-Type: application/http
9
10 POST {{alias}}/Server/ws/EndpointName/v1/Part HTTP/1.1 absolute resource path and separate Host header
11 Host: {{HOST}}
12 Content-Type: application/json
13
14 {
15   "id": "01AA11293792405383FB4A101B2AFF3E",
16   "item_number": "Part-01",
17   "name": "Part-01"
18 }
19
20 --batch_abcdeff8-fc75-4b56-8c71-56071383e77b
21 Content-Type: application/http
22
23 DELETE Part('01AA11293792405383FB4A101B2AFF3E') HTTP/1.1 resource path relative to the batch request URI
24
25
26 --batch_abcdeff8-fc75-4b56-8c71-56071383e77b-- last boundary

```

Note: The formatting of the batch request body is important. Each part must be separated by an empty line. Add an extra empty line after requests without a body (see line #4 above).

3.4.5.3 Change Sets

A **Change Set** is an atomic unit of work consisting of an unordered group of one or more [Data Modification](#) requests or [Action invocation](#) requests.

All Change Sets must meet the following requirements:

- Change Sets may not contain any GET requests or other Change Sets.
- The order of requests within a Change Set is significant; the requests within a Change Set are processed sequentially.
- If one request within a Change Set fails, then all requests in the Change Set are rolled back.
- Each Change Set must be a multipart MIME document with one part for each operation that makes up the Change Set.
- Each part representing an operation in the Change Set must include the same headers (*Content-Type* and *Content-Transfer-Encoding*) and associated values as previously

described for operations (see below).

```

1  --batch_abodeff8-fc75-4b56-8c71-56071383e77b
2  Content-Type: application/http
3
4  GET Part HTTP/1.1
5
6
7  --batch_abodeff8-fc75-4b56-8c71-56071383e77b
8  Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-a9f8-9357efb8bd ← change set boundary
9
10 --changeset_77162fcd-b8da-41ac-a9f8-9357efb8bd
11 Content-Type: application/http
12 Content-Transfer-Encoding: binary
13 Content-ID: 1
14
15 POST Part HTTP/1.1
16 Content-Type: application/json
17
18 {
19   → "id": "01AA112937924D5383FB4A101B2AFF3E",
20   → "item_number": "Part-01",
21   → "name": "Part-01"
22 }
23
24 --changeset_77162fcd-b8da-41ac-a9f8-9357efb8bd
25 Content-Type: application/http
26 Content-ID: 2
27
28 POST Part HTTP/1.1
29 Content-Type: application/json
30
31 {
32   → "id": "00BB112937924D5383FB4A101B2AFF56",
33   → "item_number": "Part-00",
34   → "name": "Part-00",
35   → "Part-ECM": [
36     → {
37       → "id": "00E01E12F3004B3283F3C88B9349E4A0",
38       → "related_id@odata.bind": "$1"
39     }
40   ]
41 }
42
43 --changeset_77162fcd-b8da-41ac-a9f8-9357efb8bd
44 Content-Type: application/http
45 Content-ID: 3
46
47 DELETE $2 HTTP/1.1
48
49
50 --changeset_77162fcd-b8da-41ac-a9f8-9357efb8bd--
51
52 --batch_abodeff8-fc75-4b56-8c71-56071383e77b
53 Content-Type: application/http
54
55 DELETE Part('01AA112937924D5383FB4A101B2AFF3E') HTTP/1.1
56
57
58 --batch_abodeff8-fc75-4b56-8c71-56071383e77b--

```

Annotations for the Change Set:

- change set boundary (points to line 8)
- first change set request (points to line 13)
- second change set request (points to line 26)
- third change set request (points to line 45)
- last boundary (end of change set) (points to line 50)

- Each request within a Change Set must specify a *Content-ID* header with a value unique within the batch request.
- Entities created by an [Insert](#) request within a Change Set can be referenced by subsequent requests within the same Change Set in places where a resource path to an existing entity can be specified. The temporary resource path for a newly inserted entity is the value of the *Content-ID* header prefixed with a \$ character. Examples of correct and wrong Change

Set requests are below.

```

1  --batch_abcdeff8-fc75-4b56-8c71-56071383e77b
2  Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
3
4  --changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
5  Content-Type: application/http
6  Content-ID: 1
7
8  POST Part: HTTP/1.1
9  Content-Type: application/json
10
11  {
12    "id": "01AA112937924D5383FB4A101B2AFF3E",
13    "item_number": "Part-01",
14    "name": "Part-01"
15  }
16
17  --changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
18  Content-Type: application/http
19  Content-ID: 1
20
21  PUT Part('01AA112937924D5383FB4A101B2AFF3E')/name: HTTP/1.1
22  Content-Type: application/json
23
24  {
25    "value": "New Part-1"
26  }
27
28  --changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
29  Content-Type: application/http
30  Content-ID: 2
31
32  GET Part.$1: HTTP/1.1
33
34  --changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
35  Content-Type: application/http
36
37  DELETE $1: Part: HTTP/1.1
38
39
40
41  --changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
42  Content-Type: multipart/mixed; boundary=changeset_98357add-c8fa-41ac-a9f8-9357efabc
43
44  --changeset_98357add-c8fa-41ac-a9f8-9357efabc
45  Content-Type: application/http
46  Content-ID: 4
47
48  POST Part: HTTP/1.1
49  Content-Type: application/json
50
51  {
52    "id": "51BC113537924D5383FB4A101B2AAA3B",
53    "item_number": "Part-02",
54    "name": "Part-02"
55  }
56
57  --changeset_98357add-c8fa-41ac-a9f8-9357efabc--
58  --changeset_77162fcd-b8da-41ac-a9f8-9357efbbd--
59
60  --batch_abcdeff8-fc75-4b56-8c71-56071383e77b--
61

```

correct Change Set request

Content-ID is not unique

wrong Change Set request

GET HTTP request is not allowed

There is no specified Content-ID

wrong Change Set request

nested Change Set request

wrong Change Set request

3.4.5.4 Responding to a Batch Request

Requests within a batch are evaluated according to the same semantics used when processing individual requests. The following response was generated by the sample batch request above.

```

--batch_abcdeff8-fc75-4b56-8c71-56071383e77b
Content-Type: application/http

HTTP/1.1 200 OK
OData-Version: 4.0
Content-
Type: application/json;odata.metadata=minimal;IEEE754Compatible=false

{

```

```

    "@odata.context":
"http://{{host}}/{{alias}}/Server/ws/EndpointName/v1/$metadata#Part",
    "value": []
}
--batch_abcdeff8-fc75-4b56-8c71-56071383e77b
Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-
a9f8-9357efbbd

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
Content-Type: application/http
Content-ID: 1

HTTP/1.1 201 Created
Preference-Applied: return=representation
Location: http://{{host}}/{{alias}}/Server/ws/EndpointName/v1/Part('01AA
112937924D5383FB4A101B2AFF3E')
OData-Version: 4.0
Content-
Type: application/json;odata.metadata=minimal;IEEE754Compatible=false

{
    "@odata.context":
"http://{{host}}/{{alias}}/Server/ws/EndpointName/v1/$metadata#Part/$en
tity",
    "id": "01AA112937924D5383FB4A101B2AFF3E",
    "major_rev": "A",
    "name": "Part-01",
    "item_number": "Part-01"
}
--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
Content-Type: application/http
Content-ID: 2

HTTP/1.1 201 Created
Preference-Applied: return=representation
Location: http://{{host}}/{{alias}}/Server/ws/EndpointName/v1/Part('00BB
112937924D5383FB4A101B2AFF56')
OData-Version: 4.0
Content-
Type: application/json;odata.metadata=minimal;IEEE754Compatible=false

{
    "@odata.context":
"http://{{host}}/{{alias}}/Server/ws/EndpointName/v1/$metadata#Part/$en
tity",
    "id": "00BB112937924D5383FB4A101B2AFF56",
    "major_rev": "A",
    "name": "Part-00",
    "item_number": "Part-00",
    "Part_BOM": [
        {
            "id": "00E01E12F3004B3283F3C88B9349E4A0",
            "sort_order": 128
        }
    ]
}

```

```
--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd
Content-Type: application/http
Content-ID: 3

HTTP/1.1 204 NoContent
OData-Version:4.0
Content-Type:text/plain

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd--
--batch_abcdeff8-fc75-4b56-8c71-56071383e77b
Content-Type: application/http

HTTP/1.1 204 NoContent
OData-Version:4.0
Content-Type:text/plain

--batch_abcdeff8-fc75-4b56-8c71-56071383e77b--
```

3.4.6 Overwriting System Properties

CWS provides the ability to overwrite the value of system properties in the Aras Innovator database. This is unique to CWS, as it is not possible to overwrite system properties with AML, the IOM, or the default OData REST API.

3.4.6.1 Override Criteria

This capability requires 3 criteria to be met to overwrite system properties with CWS:

1. The HTTP request must use the POST, PUT, or PATCH action
2. The body of the HTTP request must contain one or more system properties included in the CWS endpoint's schema. Example:
 - created_by_id
 - created_on
 - current_state
 - generation
 - is_released
 - modified_on
 - modified_by_id
 - state
 - not_lockable
3. The HTTP request is authorized by an API Key with the "Override System Properties" scope

Warning Any request that meets the criteria above will overwrite system property data in the Aras Innovator database. Only use this capability when a use case requires an application or integration to overwrite the system property values in the Aras Innovator database. This feature may replace critical metadata such as created_by_id, modified_by_id, created_on, modified_on, state, etc.

3.4.6.2 Requirements and Considerations

The following system properties have requirements or conditions to consider when overwriting their values.

- **state**: Contains the string label of the item's current Lifecycle State. Overwriting this property without also setting **current_state** will not change the label shown in the Aras Innovator client.
- **current_state**: Contains the id of the item's Lifecycle State. Overwriting this property automatically sets the **state** property.
- **generation**: Value must form a unique pair with the item's **config_id**. The database may not contain two items with the same **config_id** and **generation**.
- **is_current**: Adding a new version of an item does not automatically update the **is_current** property of the item's previous current generation. The database may not contain two items with the same **config_id** and **is_current=1**.
- **is_released**: Items with **is_released=1** should also have **not_lockable=1**. This needs to be set explicitly and is not synced automatically.
- **created_by_id**: Changing this property will affect any permissions for this item referencing the Creator identity.
- **current_state**: Items may inherit permissions from their current Lifecycle State. Changing this property may affect the permissions of the current item.

Warning Overwriting system properties may create inconsistent data in the Aras Innovator database and should be done carefully. The HTTP response from the server is not guaranteed to throw errors caused by inconsistent data.

3.4.6.3 Server Events

This capability leverages the Data Sync Service (DSS) engine to handle requests that set and update system properties. As a result, these requests will not trigger standard ItemType server events like `onAfterAdd`, `onBeforeUpdate`, etc. Instead, the DSS engine will execute `onAfterSyncAdd`, `onBeforeSyncUpdate`, etc. This provides the ability to have custom server event logic for use cases that overwrite system properties.

Refer to the Item Actions section of the **Aras Innovator 32: Data Synchronization Services Programmer's Guide** for more information.

Warning If no sync server events are events configured on the ItemType, the overwrite system property requests will not trigger any server event methods.

3.4.6.4 Permissions

Though API Keys with the "Override System Properties" scope can overwrite system properties, the user associated with the API Key must have permission to modify the data in the request. This requires configuring standard permissions, Can Add rights, MAC policies, and/or DAC policies.

3.4.6.5 Sending a Request

Overwriting system properties with CWS follows the same general process as sending other requests to the CWS endpoint. The following steps outline the general process. See the referred content for more information.

1. Refer to the section "Creating an API Key" above to create an API Key with the scope "Override System Properties"
2. Refer to the API Key section under "Request Authorization" to create an HTTP POST, PUT, or PATCH request authorized by the API Key with the "Override System Properties" scope.
3. Include one or more system properties in the body of the request

4. Send the request to the CWS endpoint

3.4.7 Uploading Files

CWS provides an upload endpoint to quickly add files to the Aras Innovator vault with a single request. This is more convenient for developers and enables integrations with systems where developers might not be able to make multiple calls to upload a file (for example, when using webhooks).

3.4.7.1 File Upload Request

The following steps outline the process of uploading a file to the vault's upload endpoint.

1. Create an **HTTP POST** request
2. Include an API Key or OAuth 2.0 access token to **authorize** the request.

Note: The user associated with the API Key or access token must have permission to upload files.

3. Add the file to the **request body** as a binary stream.

POST ▼ https://((HOST))/((alias))/vault/file?file_name=2222.pdf

Params ● Authorization ● Headers (13) ● **Body** ● Scripts ● Settings ●

☐ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☒ binary ☐ GraphQL

2222.pdf ×

Note: By default, the upload endpoint can handle a file size up to 30MB. See the Vault Configuration section below for steps to increase this limit.

4. Add the **file name** as a "file_name" query parameter.
5. Optional: Add the desired **file id** as a "file_id" query parameter. If no id is provided, one will be generated on the server.

POST ▼ https://((HOST))/((alias))/vault/file?file_name=2222.pdf&file_id=88648A471A3240EA99EA866ABB67E416

Params ● Authorization ● Headers (13) ● **Body** ● Scripts ● Settings ●

☐ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☒ binary ☐ GraphQL

2222.pdf ×

6. **Submit** the request.

3.4.7.2 File Upload Response

The vault will return a 200 (OK) HTTP status code and the file's id if the file is successfully uploaded. The following example shows a typical response:

```
{
  "file_id": <file ID or null>,
  "error_string": <error message or null>
}
```


3.4.7.3 Error Responses

Here are the error codes that may result from a request to the vault's upload endpoint. The `error_string` property in the response body also provides any error message from the vault Server.

- **413 Request Entity Too Large:** Occurs when the size of the file exceeds the limit specified in the vault configuration
- **500 Internal Server Error:** The most common causes are an invalid access token or API key, or the user does not have permission to upload files to the vault.

3.4.7.4 Vault Configuration

The following steps outline the process to increase the maximum file size accepted by the upload endpoint.

1. In the VaultServer folder, open `appsettings.json`.
2. Add the "MaxRequestBodySize" limit to the "Kestrel" section (add this section if needed).
3. Set the `MaxRequestBodySize` value (in bytes). The following example shows how to set the limit to 70MB.

```
{
  "Kestrel": {
    "Limits": {
      "MaxRequestBodySize" : 70000000
    }
  }
}
```

4. Save and close the `appsettings.json` file.
5. Open the `web.config` file in the same directory.
6. In the "system.WebServer" section, add a security section with a tag `requestLimits` and `MaxRequestBodySize` parameter.
7. Set the `MaxRequestBodySize` value (in bytes). The following example shows how to set the limit to 70MB.

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <system.webServer>
    <httpProtocol>
      // httpProtocol settings
    </httpProtocol>
    <handlers>
      // handlers
    </handlers>
    <aspNetCore>
      // aspNetCore settings
    </aspNetCore>
    <security>
      <requestFiltering>
        <requestLimits maxAllowedContentLength="70000000" />
      </requestFiltering>
    </security>
  </system.webServer>
</configuration>
```

8. Save and close the web.config file.